Unit 1 to Unit 2

All Assignments – Questions & Pseudocode

Exam-focused simple pseudocode for Assignments 1–5.

Assignment 1

1(a) – fork(), Child and Grandchild

```
START
Create a child process using fork()
IF fork fails
  Display error
  END
END IF
IF current process is Child
  Display Child PID
  Display Parent PID
  Create another process using fork()
  IF current process is Grandchild
    Display Roll Number
    Exit with status 0
  ELSE
    Wait for Grandchild
  END IF
  Exit with status 1
ELSE
  Display Parent PID
  Wait for Child
  Display process information
END IF
END
```

1(b) – fork(), wait() and utmp user information

```
START
Create a child process using fork()
IF fork fails
  Display error
  END
END IF
IF current process is Child
  Display Child PID and Parent PID
  Create Grandchild using fork()
  IF current process is Grandchild
    Read logged-in user information from utmp
    Display user information
  ELSE
    Wait for Grandchild
  END IF
ELSE
  Wait for Child
  Display Parent process information
END IF
END
```

1(c) – fork() and execv()

```
START
Create a child process using fork()
IF fork fails
  Display error
  END
END IF
IF current process is Child
  Execute user program using execv()
  IF execv fails
    Display error
    Exit with failure status
  END IF
ELSE
```

```
  Wait for Child
  Get returned status
  Display Child status
END IF
END
```

Assignment 2

2(a) – Signal handler using signal()

```
START
Define a signal handler function
Inside handler: Display signal received; Create child using fork()
IF child process
  Display Child PID
ELSE
  Display Parent PID
END IF
In main: Register handler for SIGINT
Wait for signal using pause()
END
```

2(b) – kill() system call

```
START
Take process ID as input
Take signal number as input
Send signal to process using kill()
IF kill is successful
  Display signal sent successfully
ELSE
  Display error
END IF
END
```

2(c) – pthread_create()

```
START
Define a thread function
Inside thread: Display thread information; Terminate using pthread_exit()
In main: Create thread using pthread_create()
IF thread creation fails
  Display error
ELSE
  Wait for thread to finish
END IF
END
```

Assignment 3

Keyboard Interrupt + Pipe + Fork + Fibonacci

```
START
Register signal handler for SIGINT
Wait for keyboard interrupt
WHEN SIGINT is received:
  Take N as input
  Create a pipe
  IF pipe creation fails: Display error; RETURN
  Create child using fork()
  IF fork fails: Display error; RETURN
  IF current process is Child
    Send Child PID through pipe
    Generate N Fibonacci terms
    Send each term through pipe
    Send signal ID through pipe
    Exit
  ELSE
    Read and display Child PID
    Read and display Fibonacci terms
    Read and display signal ID
    Wait for Child
  END IF
END
```

Fibonacci logic

```
START
Set first number = 0
Set second number = 1
REPEAT N times
  Display first number
  next = first number + second number
  first number = second number
  second number = next
END REPEAT
END
```

Assignment 4

File reading + fcntl() lock

```
START
Open file in read-only mode
IF file cannot be opened: Display error; END
Create struct flock
Set lock type as F_RDLCK
Set position from beginning of file
Set length for complete file
Store current process ID
Apply lock using fcntl() with F_SETLKW
IF lock unavailable: Wait until released
Display Lock acquired
Read file using read()
Display file contents
Wait for user input
Set lock type F_UNLCK
Release lock using fcntl()
Close file
END
```

Important terms

```
F_RDLCK → Read Lock
F_WRLCK → Write Lock
F_UNLCK → Unlock
F_SETLKW → Set lock and wait
F_SETLK → Set lock without waiting
F_GETLK → Check conflicting lock
read() → Read file
fcntl() → Set/release/check lock
getpid() → Get process ID
```

Assignment 5

5(A) – Named Pipe/FIFO – Program 1

```
START
Create named pipe using mknod()
Set required permissions
Open FIFO
Wait for message from Program 2
Read message from FIFO
Display received message
Create acknowledgement
Write acknowledgement into FIFO
Close FIFO
END
```

5(A) – Named Pipe/FIFO – Program 2

```
START
Take message from user
Open FIFO
Write message into FIFO
Wait for acknowledgement
Read acknowledgement from FIFO
Display acknowledgement
Close FIFO
END
```

5(B) – System V Message Queue – Server

```
START
Create/access message queue using msgget()
IF queue creation fails: Display error; END
Display Message Queue ID
Wait for message from Client
Receive using msgrcv()
Display received message
Create acknowledgement
Send using msgsnd()
Remove queue using msgctl()
END
```

5(B) – System V Message Queue – Client

```
START
Access message queue using msgget()
IF queue cannot be accessed: Display error; END
Take message from user
Set message type
Send using msgsnd()
Display Message sent
Wait for acknowledgement
Receive using msgrcv()
Display acknowledgement
END
```

Quick Pseudocode Keywords

printf() → Display

scanf() → Input

fork() → Create child process

wait() → Wait for child

execv() → Execute another program

signal() → Register signal handler

pause() → Wait for signal

kill() → Send signal

pthread_create() → Create thread

pthread_exit() → Terminate thread

pipe() → Create pipe

read() → Read data

write() → Write/send data

fcntl() → File locking

getpid() → Get process ID

mknod() → Create named pipe

msgget() → Create/access message queue

msgsnd() → Send message

msgrcv() → Receive message

msgctl() → Control/remove queue